

Export Gems via API

Export gem content in multiple formats through three distinct workflows - direct downloads, polling-based exports, and WebSocket streaming.

This guide enables automated content export for compliance, backup, distribution, and system integrations across different gem types and formats.

Export Methods Overview

Shelf provides three export workflows depending on content type and processing requirements:

Method	Use Case	Processing Time	Gem Types
Direct Download	Files ready for immediate download	Instant	Documents, Images
Polling Flow	Generated content requiring processing	10-30 seconds	Wiki PDFs, Previews
WebSocket Flow	Complex exports with real-time updates	30-60 seconds	Decision Trees

Direct Download Flow

Download files immediately without processing delays. Perfect for document attachments and images.

```
# Get signed download URL (expires in 5 minutes)
curl -X GET "https://api.shelf.io/content/v1/gems/acme-gem-id-123/attachment?skipRedirect=true" \
  -H "Authorization: $SHELF_API_TOKEN"
```

Response with download URL:

```
{  
  "url": "https://s3.us-east-1.amazonaws.com/gemshelf/...",  
  "expiresAt": "2025-08-14T16:30:00Z"  
}
```

Direct download with automatic redirect:

```
# Download file immediately  
curl -X GET "https://api.shelf.io/content/v1/gems/acme-gem-id-  
123/attachment" \  
  -H "Authorization: $SHELF_API_TOKEN" \  
  -L -o "acme-financial-policy.pdf"
```

Advanced Options:

```
# Extended expiry (up to 60 minutes)  
curl -X GET "https://api.shelf.io/content/v1/gems/acme-gem-id-  
123/attachment?expiresInMinutes=60" \  
  -H "Authorization: $SHELF_API_TOKEN" \  
  -L -o "document-extended.pdf"  
  
# Specific version  
curl -X GET "https://api.shelf.io/content/v1/gems/acme-gem-id-  
123/attachment?version=v2" \  
  -H "Authorization: $SHELF_API_TOKEN" \  
  -L -o "document-v2.pdf"
```

❗ Download URLs expire in 5 minutes by default. Request new URLs if downloads fail due to expiration.

Polling Flow

Request processing jobs and poll for completion. Used for Wiki page PDFs and generated previews.

Wiki Page HTML Export

```
# Immediate HTML export
curl -X GET "https://api.shelf.io/content/v1/gems/wiki-page-id-456/wiki-content" \
  -H "Authorization: $SHELF_API_TOKEN" \
  -o "acme-wiki-page.html"
```

Wiki Page PDF Generation

1 Request PDF Generation

```
EXPORT_ID=$(curl -s -X GET "https://api.shelf.io/previews/wiki-page-id-456/wiki/pdf/request" \
  -H "Authorization: $SHELF_API_TOKEN" | jq -r '.exportId')
```

Response with job ID:

```
{
  "exportId": "export_abc123def456"
}
```

2 Poll and Download

```
# Get download URL when ready
DOWNLOAD_URL=$(curl -s -X GET "https://api.shelf.io/previews/wiki-page-id-456/wiki/pdf/download-url?exportId=$EXPORT_ID" \
  -H "Authorization: $SHELF_API_TOKEN" | jq -r '.url')

# Download generated PDF
curl -L "$DOWNLOAD_URL" -o "acme-wiki-export.pdf"
```

PDF Preview Generation

```
# Generate PDF preview for any attachment
curl -X GET "https://api.shelf.io/content/v1/gems/acme-gem-id-123/attachment/preview/pdf" \
  -H "Authorization: $SHELF_API_TOKEN" \
  -L -o "document-preview.pdf"
```

WebSocket Flow

Real-time export processing for complex content like decision trees. Requires WebSocket client implementation.

Get WebSocket Token

```
WS_TOKEN=$(curl -s 'https://api.shelf.io/auth/ws/one-time-tokens/' \
-H "Authorization: $SHELF_API_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "context": {
    "meta": {
      "connectionSource": "previews",
      "purpose": "generate-decision-tree-pdf"
    },
    "data": {}
  }
}' | jq -r '.wsToken')
```

Decision Tree Exports

Visual Diagram PDF:

```
# Connect to WebSocket and send generation request
echo '{
  "action": "send_preview_generation",
  "payload": {
    "gemId": "decision-tree-id-789",
    "type": "generate-decision-tree-diagram-pdf"
  }
}' | wscat -c "wss://api.ws.shelf.io/previews?wsToken=$WS_TOKEN"
```

Complete Content PDF:

```
# Full decision tree content export
echo '{
  "action": "send_preview_generation",
  "payload": {
    "gemId": "decision-tree-id-789",
    "type": "generate-decision-tree-content-pdf"
  }
}' | wscat -c "wss://api.ws.shelf.io/previews?wsToken=$WS_TOKEN"
```

Step Data Export

```
# Export individual decision tree step
curl -X GET "https://api.shelf.io/decision-tree/step?gemId=decision-
tree-id-789&stepId=step-001" \
  -H "Authorization: $SHELF_API_TOKEN" \
  -o "decision-step-data.json"
```

Example JSON response:

```
{
  "steps": [
    {
      "stepId": "step-001",
      "question": "What type of loan are you applying for?",
      "answerOptions": [
        {
          "id": "opt-001",
          "text": "Personal Loan",
          "nextStepId": "step-002"
        }
      ]
    }
  ],
  "metadata": {
    "totalSteps": 8,
    "treeVersion": 1
  }
}
```

i Decision Tree exports require WebSocket client implementation. Consider using the Shelf web interface for complex decision tree exports.

Bulk Exports

Export multiple gems as ZIP archives using polling workflow.

1 Create Export Job

```
JOB_ID=$(curl -s -X POST
"https://api.shelf.io/gems/attachments/export-jobs" \
-H "Authorization: $SHELF_API_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "gemIds": [
    "acme-gem-id-123",
    "acme-gem-id-456",
    "acme-gem-id-789"
  ]
}' | jq -r '.exportJobId')
```

Response with job details:

```
{
  "exportJobId": "job_bulk_export_001",
  "status": "processing",
  "itemCount": 3
}
```

2 Monitor and Download

```
# Poll for completion and get download URL
DOWNLOAD_URL=$(curl -s -X GET
"https://api.shelf.io/gems/attachments/export-jobs/$JOB_ID" \
-H "Authorization: $SHELF_API_TOKEN" | jq -r '.url')

# Download ZIP archive
curl -L "$DOWNLOAD_URL" -o "bulk-attachments.zip"
```

! Bulk exports only include gems with downloadable attachments. Wiki pages and decision trees require separate export workflows.

Error Handling

Permission Errors

403 Forbidden - Can't View Gem

```
{
  "error": {
    "code": "CANT_VIEW_GEM",
    "status": 403,
    "message": "You can't view this gem"
  }
}
```

Solution: Ensure your API token has read access to the gem and its parent collection through appropriate User Group permissions.

Content Errors

422 Unprocessable Entity - No Content

```
{
  "error": {
    "status": 422,
    "message": "Gem has no content to download"
  }
}
```

Solution: Use the correct export method for the gem type. Wiki pages need `wiki-content` endpoint, not `attachment` endpoint.

Expiration Errors

410 Gone - URL Expired

HTTP Status: 410 Gone

Solution: Download URLs expire after the specified time. Request a new export URL and download immediately.

Size Limit Errors

413 Payload Too Large

```
{
  "error": {
    "message": "Bulk export exceeds size limit (500MB)",
    "status": 413
  }
}
```

Solution: Reduce the number of gems in bulk exports or use individual exports for large files.

- ✓ **Processing Times:** Direct downloads are instant. PDF generation takes 10-30 seconds. Bulk exports may take 1-5 minutes depending on content size.